Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- Beam Search
- How it works
- Real-World Applications
- Benefits
- Challenges and Limitations
- Practice code

Text Summarization

- When You DO Need an Encoder :

- Abstractive Summarization: When you want the model to produce human-like summaries that may use new words or paraphrasing.
- High-Quality/Complex Text: For complex, technical, or long documents, an encoder helps ensure the summary is accurate and relevant.
- Using Architectures like T5/BART: These models are designed specifically to use an encoder to understand the input and a decoder to generate the output

Text Summarization

- When Encoders Might Not Be Needed :
- Extractive Summarization: Simple extractive methods, such as TextRank or selecting top-N sentences based on frequency, do not need a deep learning encoder model.
- Decoder-Only LLMs: Modern large language models (like GPT-4 or Gemini) are decoder-only. They can handle summarization through prompting. However, the model effectively "encodes" the prompt internally, even without a separate encoder architecture.

Beam search

- Issue of why greedy decoding fails, how beam search balances exploration and exploitation, and its limitations.

- Greedy Search: Takes the argmax at every step. It is fast but short-sighted. It traps the model in a local optimum because it cannot backtrack if a high-probability initial token leads to a low-probability sequence later.

- Exhaustive Search: Finds the true global optimum but is computationally infeasible.

Beam Search

- finding the optimal solution to complex problems often involves navigating vast search spaces. Traditional search methods like depth-first and breadth-first searches have limitations, especially when it comes to efficiency and memory usage. This is where the Beam Search algorithm comes into play. Beam Search is a heuristic-based approach that offers a middle ground between these traditional methods by balancing memory consumption and solution optimality.

Heuristic Techniques

- Heuristic techniques are strategies that utilize specific criteria to determine the most effective approach among multiple options for achieving a desired goal. These techniques enhance the efficiency of search processes by prioritizing speed over systematic and exhaustive exploration. Heuristics are particularly useful for solving complex problems, such as the traveling salesman problem, in a time-efficient manner, often yielding good solutions without the exhaustive computational cost of exponential-time algorithms.

Beam Search Algorithm

- Beam Search is a heuristic search algorithm that navigates a graph by systematically expanding the most promising nodes within a constrained set. This approach combines elements of breadth-first search to construct its search tree by generating all successors at each level. However, it only evaluates and expands a set number, W, of the best nodes at each level, based on their heuristic values. This selection process is repeated at each level of the tree.

Applications of Beam Search Algorithm

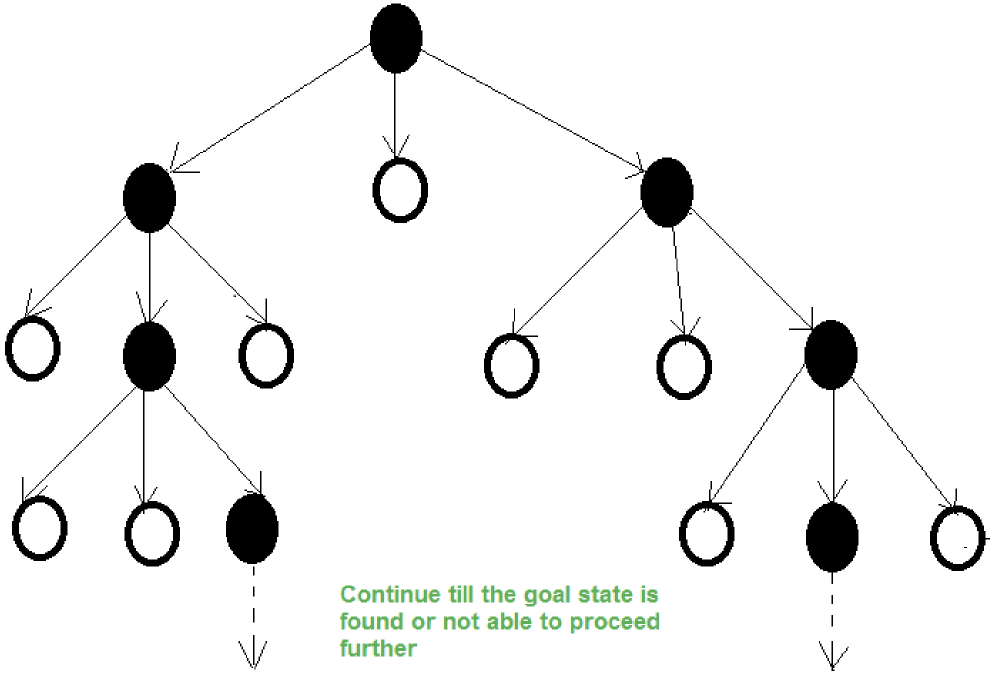
- **Natural Language Processing (NLP):** For tasks like machine translation and speech recognition where the goal is to find the best sequence of words.
- **Robotics:** In pathfinding algorithms where a robot must find an efficient path in an environment.
- **Game AI:** In strategic games where it is impractical to explore every possible move due to the enormous search space.

Characteristics of Beam Search

- **Width of the Beam (W):** This parameter defines the number of nodes considered at each level. The beam width W directly influences the number of nodes evaluated and hence the breadth of the search.
- **Branching Factor (B):** If B is the branching factor, the algorithm evaluates $W \times B$ nodes at every depth but selects only W for further expansion.
- **Completeness and Optimality:** The restrictive nature of beam search, due to a limited beam width, can compromise its ability to find the best solution as it may prune potentially optimal paths.
- **Memory Efficiency:** The beam width bounds the memory required for the search, making beam search suitable for resource-constrained environments.

Characteristics of Beam Search

- **Example:** Consider a search tree where W = 2 and B = 3. Only two nodes (black nodes) are selected based on their heuristic values for further expansion at each level.



Beam Search

How Beam Search Works

- **Initialization**: Start with the root node and generate its successors.
- **Node Expansion**: From the current nodes, generate successors and apply the heuristic function to evaluate them.
- **Selection**: Select the top W nodes according to the heuristic values. These selected nodes form the next level to explore.
- **Iteration**: Repeat the process of expansion and selection for the new level of nodes until the goal is reached or a certain condition is met (like a maximum number of levels).
- **Termination**: The search stops when the goal is found or when no more nodes are available to expand.

How Beam Search Works

- Instead of one path (Greedy) or all paths (Exhaustive), we maintain *k* active candidates (the "beam width").

1. **Initialization:** Start with the special token `<SOS>` as a single path with score 0.
2. **Expansion:** For each of the k paths, calculate the probability of the next token for all V possibilities.
3. **Scoring:** We are calculating $P(y_1, ..., y_t) = \prod_{i=1}^{t} P(y_i|y_{<i})$. In practice, we use log-probabilities: $\sum_{i=1}^{t} \log P(y_i|y_{<i})$.
4. **Pruning:** Flatten the list of all $k \times V$ candidates and select the top k based on the cumulative log-probability.
5. **Termination:** Stop when `<EOS>` is generated or the maximum length is reached.

Advantages of Beam Search

- **Efficiency**: By limiting the number of nodes expanded, Beam Search can navigate large search spaces more efficiently than exhaustive searches.
- **Flexibility**: The algorithm can be adjusted for different problems by modifying the beam width and heuristic function.
- **Scalability**: Suitable for problems where the solution paths are vast and complex, as it does not require all nodes to be stored in memory.

- Complexity : O(T , V , k), where T is the sequence length, V is vocabulary size, and k is the beam width.
- k is a hyperparameter. k=1 is Greedy Search; k-> infinity approaches Exhaustive Search.

Limitations of Beam Search

- **Suboptimality**: There is no guarantee that Beam Search will find the optimal solution, especially if the beam width is too narrow.
- **Heuristic Dependency**: The effectiveness of Beam Search is highly dependent on the quality of the heuristic function. Poor heuristics can lead to suboptimal searching and results.
- Beam search is inherently biased towards "safe," high-probability phrases. This often results in generic or repetitive output in open-ended text generation.

Modern Alternatives

- Top-k Sampling: Instead of taking the top-k by score, sample randomly from the top-k probability distribution.
- Top-p (Nucleus) Sampling: Sample from the smallest set of tokens whose cumulative probability exceeds p.
- Diversity Penalty: Subtracting a penalty score if the beam includes tokens already generated in other active beams.

Summary

- Beam search is the industry standard for deterministic generation tasks (Machine Translation, Summarization), while Sampling is preferred for "creative" tasks (Storytelling, Chatbots).

Translation example

- **Source (Korean):** "어제 본 영화는 생각보다 훨씬 더 감동적이었어요."
- **Literal Meaning:** The movie I saw yesterday was much more moving than I thought.

Decoding Strategy Comparison

Strategy	Output Sequence	Analysis
Greedy Search $(k = 1)$	"The movie I saw yesterday was good."	**Short-sighted.** It picked the most frequent word "good" early on and reached a high-probability stop point quickly. It loses the nuance of "moving" and "more than I thought."
Beam Search $(k = 5)$	"The movie I saw yesterday was much more moving than I expected."	**Optimal & Fluent.** By keeping 5 hypotheses, it bypassed the generic "good" for the higher-quality but lower-initial-probability path. It captured the comparative structure and the emotional weight accurately.

Decoding Strategy Comparison

Top-p (Nucleus) Sampling	"That film yesterday was way more touching than I ever imagined."	**Creative & Diverse.** By sampling from the probability mass, it used more varied vocabulary ("film," "touching," "imagined"). It sounds more natural/human but carries a slight risk of losing exact fidelity.

Exercise

- Greedy vs Beam (in Uzbek language and English)
- Apply length penalty. (not more than three words)

Reference

- https://www.geeksforgeeks.org/machine-learning/introduction-to-beam-search-algorithm/

Question

Thank you